



HASHDIT

HashDit

Smart Contract Security

ListaDao Mainnet MarketFactory & ListaRevenueDistributor Security Audit Report

ListaDao

09 Jul 2026 - 10 Jul 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	09 Jul 2026– 10 Jul 2026
Overall Risk	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-token/pull/111 https://github.com/lista-dao/moolah/pull/206
Commit	feature/revenue-distributor-eth-support feature/eth-market-factory

Smart Contract List

No.	Contract Name	Verdict	Details
1	MarketFactory.sol	Informational	[I03] [I04]
2	ListaRevenueDistributor.sol	Informational	[I01] [I02] [I05] [I06]
3	deploy_marketFactory.sol	Informational	[I07] [I08] [I09]
4	deploy_listaRevenueDistributo r.ts	Informational	[I08]

Findings

[I01] Misleading RevenueDistributed events when targets are address(0)

Contract ListaRevenueDistributor.sol

Severity Informational

Description	Description The <code>_distributeToken</code> and <code>_distributeTokenWithCost</code> functions in <code>ListaRevenueDistributor</code> emit <code>RevenueDistributed(token, amount0, amount1)</code> using the computed split amounts, but they skip the actual transfer whenever the corresponding target is <code>address(0)</code>. <pre>JavaScript emit RevenueDistributed(token, amount0, amount1); emit RevenueDistributedWithCost(token, amount0, amount1, cost, cost, tokenCostToAddress);</pre>
	Impact As a result, the event can report <code>amount0</code> and <code>amount1</code> as distributed even though no tokens were moved. On ETH, where all targets are zero, every <code>distributeTokens</code> call would emit the full <code>amount0</code> and <code>amount1</code> while transferring nothing, since <code>emergencyWithdraw</code> is expected to be called instead to allow the cross chain bridge to BSC for buyback. Off chain accounting and bridging bots that rely on these events will over count the funds that were actually distributed. Emit the amounts that were actually transferred rather than the computed split amounts, so the event reflects real token movement.
Recommendation	Alternatively, revert or short circuit the distribution when all targets are <code>address(0)</code> to avoid emitting misleading <code>RevenueDistributed</code> events, delegating the actual event to within the <code>emergencyWithdraw</code> function instead, since tokens are now manually distributed.
Status	Fixed, not initializes an <code>actualAmount0</code> and <code>actualAmount1</code> for the <code>RevenueDistributed</code> and <code>RevenueDistributedWithCost</code> within <code>_distributeToken</code> and <code>_distributeTokenWithCost</code> that emits the amount transferred only when the relevant distribution addresses are non-zero.

[I02] Missing event for emergencyWithdraw

Contract	ListaRevenueDistributor.sol
Severity	Informational
Description	Description The ListaRevenueDistributor.emergencyWithdraw function moves funds out of the contract but does not emit any event when it does so. <pre> JavaScript /** * @dev emergency withdraw tokens from contract * @param _token token address * @param _to receiver address * @param _amount token amount */ function emergencyWithdraw(address _token, address _to, uint256 _amount) external onlyRole(MANAGER) { require(_to != address(0), "to is the zero address"); require(_amount > 0, "Invalid amount"); IERC20(_token).safeTransfer(_to, _amount); } </pre> Impact This is inconsistent with LendingFeeRecipient.emergencyWithdraw, which emits an EmergencyWithdraw event for the same type of action. Without an emitted event, these bridging withdrawals cannot be easily tracked or verified on chain, reducing auditability and making it harder for off chain monitoring to detect and record sensitive fund movements.
Recommendation	Add an EmergencyWithdraw event to ListaRevenueDistributor.emergencyWithdraw and emit it whenever funds are withdrawn, including relevant details such as the token, recipient, and amount.
Status	Fixed, now emits the EmergencyWithdrawn event when emergencyWithdraw is called.

[I03] setRateCalculator and setBrokerLiquidator use a role that conflicts with documentation and provide no way to unset

Contract	MarketFactory.sol
Severity	Informational
Description	Description The setRateCalculator and setBrokerLiquidator functions in MarketFactory are restricted to DEFAULT_ADMIN_ROLE, but the PR text states these setters should be controlled by OPERATOR. This mismatch between the code and the documentation should be resolved so the intended access role is clear. In addition, both setters enforce require(!= address(0)), which means that once an address is set it can never be reset back to zero.

This is functionally acceptable, but it removes the ability to disable the fixed term infrastructure without performing a full upgrade.

JavaScript

```
/**
 * @dev Set the rate calculator address
 * @param _rateCalculator The address of the rate calculator contract
 */
function setRateCalculator(address _rateCalculator) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_rateCalculator != address(0), "ZeroAddress");
    emit RateCalculatorUpdated(address(rateCalculator),
_rateCalculator);
    rateCalculator = IRateCalculator(_rateCalculator);
}

/**
 * @dev Set the broker liquidator address
 * @param _brokerLiquidator The address of the broker liquidator
contract
 */
function setBrokerLiquidator(address _brokerLiquidator) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_brokerLiquidator != address(0), "ZeroAddress");
    emit BrokerLiquidatorUpdated(address(brokerLiquidator),
_brokerLiquidator);
    brokerLiquidator = IBrokerLiquidator(_brokerLiquidator);
}
```

Recommendation	Confirm which role is intended to gate setRateCalculator and setBrokerLiquidator, and align either the code or the documentation so they match.
Status	Fixed, the setRateCalculator and setBrokerLiquidator now indicated within the docs to be called by the DEFAULT_ADMIN_ROLE.

[I04] MarketFactory to ListaRevenueDistributor role dependencies

Contract	MarketFactory.sol
-----------------	-------------------

Severity	Informational
-----------------	---------------

Description	Description When revenueDistributor is non zero, _createMarket calls revenueDistributor.addTokensToWhitelist(), which is restricted by onlyRole(DEFAULT_ADMIN_ROLE) on ListaRevenueDistributor.
--------------------	--

JavaScript

```
/**
 * @dev Set the rate calculator address
 * @param _rateCalculator The address of the rate calculator contract
 */
function setRateCalculator(address _rateCalculator) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_rateCalculator != address(0), "ZeroAddress");
    emit RateCalculatorUpdated(address(rateCalculator),
_rateCalculator);
    rateCalculator = IRateCalculator(_rateCalculator);
}

/**
 * @dev Set the broker liquidator address
 * @param _brokerLiquidator The address of the broker liquidator
contract
 */
function setBrokerLiquidator(address _brokerLiquidator) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_brokerLiquidator != address(0), "ZeroAddress");
    emit BrokerLiquidatorUpdated(address(brokerLiquidator),
_brokerLiquidator);
    brokerLiquidator = IBrokerLiquidator(_brokerLiquidator);
}
```

Impact

On ETH, the ListaRevenueDistributor deployment grants the admin role only to the deployer and the MANAGER role to the Safe, and it never grants MarketFactory the admin role. The MarketFactory deploy script contains a TODO to fill in the revenueDistributor address. If that address is populated without also granting MarketFactory the DEFAULT_ADMIN_ROLE on the distributor, then every createMarket and batchCreateMarkets call will revert because the whitelist step fails the role check.

Recommendation

Either leave revenueDistributor = address(0) in the MarketFactory constructor on ETH, in which case the guard address(revenueDistributor) != address(0) cleanly skips the whitelist step, which is acceptable because ETH uses emergencyWithdraw rather than distributeTokens and does not need the distributor whitelist, or add an explicit step to the deployment runbook that grants MarketFactory the DEFAULT_ADMIN_ROLE on ListaRevenueDistributor before setting the address

Status

Acknowledged, ListaDao has indicated within documentation that an explicit step where the deployer EOA grants DEFAULT_ADMIN_ROLE to the MarketFactory on ListaRevenueDistributor will be performed before any createMarket call is made.

[I05] emergencyWithdraw name does not reflect its routine use for manual token extraction

Contract ListaRevenueDistributor.sol

Severity	Informational
Description	Description The emergencyWithdraw function in ListaRevenueDistributor is named as if it were reserved for emergency situations, but its intended use is routine. The MANAGER is expected to call it to manually extract tokens as part of the normal cross chain bridge flow to BSC for buyback if the listaDistributeToAddress, autoBuybackAddress and revenueWalletAddress is the zero address. <pre> JavaScript /** * @dev emergency withdraw tokens from contract * @param _token token address * @param _to receiver address * @param _amount token amount */ function emergencyWithdraw(address _token, address _to, uint256 _amount) external onlyRole(MANAGER) { require(_to != address(0), "to is the zero address"); require(_amount > 0, "Invalid amount"); IERC20(_token).safeTransfer(_to, _amount); } </pre> Impact The current name misrepresents the function's purpose and can mislead readers, integrators, and monitoring systems into treating a regular operational action as an emergency event.
Recommendation	Rename the function to something that accurately describes its intended use, such as a manual withdrawal used for bridging tokens to BSC for buyback. Choosing a name aligned with the actual workflow improves code clarity and prevents confusion about when the function is meant to be used.
Status	Acknowledged, this naming is retained to allow the LendingFeeRecipient contract to potentially perform a emergencyWithdraw call towards the ListaRevenueDistributor (although not recommended unless appropriate access control is in place). Additionally, an EMERGENCY_WITHDRAWER role is added as a fix for [M01].

[I06] Delegating ETH revenue distribution to manual MANAGER withdrawals removes on-chain split enforcement

Contract	ListaRevenueDistributor.sol
Severity	Informational
Description	Description

On ETH the protocol routes revenue out through manual **MANAGER** actions, calling `emergencyWithdraw` and then bridging to BSC for buyback, rather than using the automated on-chain distribution path. Because the funds involved are protocol fee revenue and not user deposits or collateral, this does not put depositor or borrower principal at risk. It does, however, introduce several disadvantages that mainly affect LISTA tokenholders and protocol transparency.

Impact

1. First, the code enforced revenue split is lost. The automated path in `_distributeToken` applies the `distributeRate` split between the buyback address and the revenue wallet, while `emergencyWithdraw` enforces no ratio and sends an arbitrary amount to an arbitrary address. The proportion of revenue that tokenholders expect to flow to LISTA buyback versus treasury is therefore no longer guaranteed on chain and depends entirely on **MANAGER** executing correctly each time.
2. Second, `emergencyWithdraw` has no destination whitelist. Unlike the `distributeWithCost` path, which restricts cost transfers to `tokenCostToAddress` or a whitelisted `costTo` address, `emergencyWithdraw` only checks that the receiver is not `address(0)`. A single compromised or misbehaving **MANAGER** key can redirect the entire revenue stream to any address with no on chain constraint.
3. Third, buyback becomes irregular and bridge dependent. Automated `autoBuyback` buys LISTA continuously as revenue arrives, while manual bridging happens only when **MANAGER** acts, on an ad hoc cadence, and only after funds survive a cross chain bridge hop. **MANAGER** effectively market times the buyback, so LISTA holders bear execution timing and bridge risk that they would not bear under continuous automation.

Recommendation

At minimum, add an event to `ListaRevenueDistributor.emergencyWithdraw` so that manual withdrawals are recorded on chain, and give it a destination whitelist or introduce a dedicated named withdrawal function.

Status

Acknowledged, this design was utilized due to the absence of the LISTA token, `autoBuyback` contract, and revenue wallet on ETH mainnet.

Note that with the deployment of the LISTA OFT token on mainnet, mainnet revenue distribution could be possible, but this is at the discretion of ListaDao

[I07] TODO for the ETH mainnet lista address in `deploy_marketFactory.sol`

Contract `deploy_listaRevenueDistributor.ts`

Severity **Informational**

Description **Description**

The deployment script in `deploy_marketFactory.sol` contains an unresolved TODO to supply the ETH mainnet lista address. Leaving this placeholder unresolved means the script may deploy with a missing or incorrect lista address on mainnet, resulting in a misconfigured deployment. Configuration values that are left as TODOs are easy to overlook during deployment and can cause the contract to reference the wrong address in production.

JavaScript

```
const listaAddress = '0xFceB31A79F71AC9CBDCF853519c1b12D379EdC46'; //  
TODO: replace with LISTA ETH mainnet address once deployed (initialize  
requires non-zero)
```

Recommendation	Ensure that the TODO is resolved by filling in the correct ETH mainnet lista address before deploying, and verify the value against the official mainnet deployment.
-----------------------	--

Status	Acknowledged, ListaDao has indicated that once mainnet LISTA token is deployed, the address will be adjusted and resolved appropriately if required.
---------------	--

[I08] Admin and upgrade authority remain on the deployer EOA with no role or ownership transfer step

Contract	<code>deploy_marketFactory.sol</code> & <code>deploy_listaRevenueDistributor</code>
-----------------	---

Severity	Informational
-----------------	---------------

Description	Description Both mainnet contracts finish deployment fully under the control of a single deployer EOA, and neither ETH script includes a follow up step to move control to a Safe or timelock. Impact <code>deploy_marketFactory.sol</code> calls <code>initialize(deployer, operator, pauser)</code>, so the deployer EOA holds <code>DEFAULT_ADMIN_ROLE</code>, which is the authority for <code>_authorizeUpgrade</code> (UUPS), <code>setRateCalculator</code>, and <code>setBrokerLiquidator</code>. There is no <code>deploy_marketFactory_transferRole.sol</code> in <code>script/eth/</code>, where only <code>deploy_marketFactory.sol</code> and <code>deploy_createMarket.sol</code> exist. This is unlike every other ETH module, which ships a <code>*_transferRole.sol</code> that grants admin to the timelock and revokes the deployer. <code>deploy_listaRevenueDistributor.ts</code> sets <code>admin = deployer</code>. Because the contract is a transparent proxy (it is <code>AccessControlUpgradeable</code>, not UUPS), <code>hre.upgrades.deployProxy</code> also deploys a <code>ProxyAdmin</code> owned by the deployer EOA. As a result, both the <code>DEFAULT_ADMIN_ROLE</code> and the upgrade key sit on one EOA. A <code>transfer_role.ts</code> exists in the repo but is not wired into this ETH flow.
--------------------	--

	The impact is that on mainnet, one hot EOA can upgrade the revenue contract, drain or redirect configuration, and through admin grant itself any role. This concentrates critical control in a single hot key and creates a significant point of failure.
Recommendation	Consider adding an explicit, role granting to the timelock or Safe as well as role revoking for the deployer as part of the deployment runbook atomically.
Status	Acknowledged, ListaDao has indicated that dedicated deployment steps will be performed to ensure the deployer EOA does not retain sensitive permissioned roles.

[I09] Incomplete role configuration in the MarketFactory deployment script prevents market creation	
Contract	deploy_marketFactory.sol
Severity	Informational
Description	Description The ETH deployment script for MarketFactory, deploy_marketFactory.sol, deploys and initializes the contract but never grants MarketFactory the roles it requires on the external contracts it calls during market creation, and no separate ETH configuration script performs those grants. • When creating a market, MarketFactory is the caller into Moolah, Liquidator, PublicLiquidator, and optionally ListaRevenueDistributor. • On Moolah, createMarket requires the OPERATOR role, while batchToggleLiquidationWhitelist, setProvider, setWhiteList, setFlashLoanTokenBlacklist, and setMarketBroker require the MANAGER role. • On Liquidator, setMarketWhitelist, setTokenWhitelist, and batchSetSmartProviders require MANAGER • On PublicLiquidator batchSetSmartProviders requires MANAGER. • If the revenue distributor address is wired in, addTokensToWhitelist requires DEFAULT_ADMIN_ROLE on ListaRevenueDistributor. Impact Because none of these roles are granted, the deployed MarketFactory cannot create any market: every createMarket and batchCreateMarkets call reverts on the first role gated external call it reaches.
Recommendation	Consider adding an ETH configuration script, or explicit and verified runbook steps, that after deployment grants MarketFactory the OPERATOR and MANAGER roles on Moolah, the MANAGER role on Liquidator and PublicLiquidator, and DEFAULT_ADMIN_ROLE on ListaRevenueDistributor whenever that address is configured
Status	Acknowledged, ListaDao has indicated that appropriate deployment steps will be taken to grant necessary roles for the MarketFactory when mainnet market creation is ready.

Internal Findings

Note: This findings were self-discovered internally by ListaDao.

[M01] emergencyWithdraw reuses MANAGER role, BSC bot gains arbitrary withdrawal on upgrade

Contract ListaRevenueDistributor.sol

Severity Medium

Description

emergencyWithdraw was gated by onlyRole(MANAGER). On BSC mainnet, MANAGER is held by bot hot wallet 0x44CA...F677 (verified on-chain: hasRole(MANAGER, 0x44CA...F677) = true on proxy 0x34B504...fa3Da). If BSC ever upgrades to this implementation, the bot's permission semantics silently expand from "trigger pre-configured distribution (harmless)" to "withdraw any token to any address (full fund access)" — without any explicit role grant.

JavaScript

```
/**
 * @dev emergency withdraw tokens from contract
 * @param _token token address
 * @param _to receiver address
 * @param _amount token amount
 */
function emergencyWithdraw(address _token, address _to, uint256
_amount) external onlyRole(MANAGER) {
    require(_to != address(0), "to is the zero address");
    require(_amount > 0, "Invalid amount");
    IERC20(_token).safeTransfer(_to, _amount);
}
```

Description

This is because Proxy storage (including role mappings) persists across implementation upgrades. The bot retains MANAGER, but MANAGER's callable function set now includes emergencyWithdraw.

Impact

emergencyWithdraw was gated by onlyRole(MANAGER). On BSC mainnet, MANAGER is held by bot hot wallet 0x44CA...F677 (verified on-chain: hasRole(MANAGER, 0x44CA...F677) = true on proxy 0x34B504...fa3Da). If BSC ever upgrades to this implementation, the bot's permission semantics silently expand from "trigger pre-configured distribution (harmless)" to "withdraw any token to any address (full fund access)" — without any explicit role grant.

Recommendation	Introduced a dedicated EMERGENCY_WITHDRAWER role (bytes32 public constant, no storage slot – upgrade-safe). emergencyWithdraw now requires onlyRole(EMERGENCY_WITHDRAWER). This role is only granted to Manager Safe (multisig) and never to bot wallets. If BSC upgrades to this implementation, the bot cannot call emergencyWithdraw because it only holds MANAGER, not EMERGENCY_WITHDRAWER.
Status	Fixed, the existing EOA holding the MANAGER role allowed to perform emergencyWithdraw can no longer pull funds from ListaRevenueDistributor if BSC ever upgrades to the mainnet ListaRevenueDistributor implementation contract. It is recommended not to perform this upgrade unless bridging from BSC → ETH is required.